

Olympics Medals Aggregator Improvements

```
'''
    Olympics Medals
    Author: Saksham Rath
'''

from argparse import ArgumentParser as ap
import os

parser = ap()
parser.add_argument('--path', type=str, required = True)
args = parser.parse_args()

# dictionary for the data
totalData = {}

# looping through the directory
for fileName in os.listdir(args.path):
    # read the file
    with open(os.path.join(args.path, fileName), "r") as f:
        for line in f:
            data = line.strip().split("-")
            # print(data)
            if not data[0] in totalData.keys():
                totalData[data[0]] = [0,0,0]
            # loop through data of file and set the values for the
            data
            for i in range(3):
                totalData[data[0]][i] += int(data[i+1])

# sort as per gold medals and break ties lexicographically
sorted_data = dict(sorted(totalData.items(), key=lambda
item: (-item[1][0], item[0])))
print(sorted_data)
```

Your script is a neat and functional way to aggregate Olympic medals from multiple files. Here's a breakdown of what it's doing and a few improvements or suggestions for clarity, safety, and style:

What the script does:

- Accepts a directory path via command line using `--path`.
 - Iterates over all files in the specified directory.
 - Each file line is expected to have the format:
`Country-Gold-Silver-Bronze` (e.g., `India-1-2-3`).
 - Accumulates medal counts for each country.
 - Sorts the countries:
 - First by **gold medals in descending** order.
 - Then **lexicographically** by country name in case of ties.
 - Prints the final dictionary of countries and their medal tallies.
-

What's working well:

- Good use of `argparse` for CLI.
 - Handles multiple files.
 - Correct sorting logic.
 - Clean data structure.
-

Suggested Improvements:

1. **Robustness:** Check if a file is actually a text file.
 2. **Safety:** Add error handling for bad formats.
 3. **Readability:** Improve variable names and comment clarity.
 4. **Output:** Consider formatting the output better for readability.
-

Improved version:

```
python
```

```

'''
    Olympics Medals
    Author: Saksham Rathi
'''

import os
from argparse import ArgumentParser

# Setup argument parser
parser = ArgumentParser(description="Aggregate Olympic medals from files.")
parser.add_argument('--path', type=str, required=True, help="Directory path containing medal files")
args = parser.parse_args()

medal_data = {}

# Iterate over files in the directory
for filename in os.listdir(args.path):
    file_path = os.path.join(args.path, filename)

    # Make sure it's a file
    if os.path.isfile(file_path):
        with open(file_path, "r") as file:
            for line in file:
                parts = line.strip().split("-")
                if len(parts) != 4:
                    print(f"Skipping malformed line in {filename}: {line.strip()}")
                    continue

                country, *medals = parts
                try:
                    medals = list(map(int, medals))
                except ValueError:
                    print(f"Invalid medal numbers for {country} in {filename}. Skipping.")
                    continue

                if country not in medal_data:
                    medal_data[country] = [0, 0, 0]
                for i in range(3):
                    medal_data[country][i] += medals[i]

```

```
# Sort primarily by gold, then by country name
sorted_medals = dict(sorted(medal_data.items(), key=lambda item: (-item[1][0],
item[0])))

# Print in a nice format
print("Country\tGold\tSilver\tBronze")
for country, counts in sorted_medals.items():
    print(f"{country}\t{counts[0]}\t{counts[1]}\t{counts[2]}")
```

Let me know if you want to output it to a CSV or JSON file, or build a web UI around it!

↳ item\{0\}

what if here in case of tie we want to print in reverse lexicographical order ?

Great question! If you want to break ties in **reverse lexicographical order** (i.e., "Zimbabwe" comes before "Argentina" in case of a tie in gold medals), you just need to **reverse the second key** in the `sorted()` function.

Currently, you have:

```
python

sorted(totalData.items(), key=lambda item: (-item[1][0], item[0]))
```

To sort in reverse lexicographical order instead of normal lexicographical order, change `item[0]` to `-item[0]` **won't work** because `item[0]` is a string. Instead, you can negate the string by using `reverse` in a clever way:

✓ Change it to:

```
python

sorted(totalData.items(), key=lambda item: (-item[1][0], -ord(item[0][0])))
```

Wait — that only handles the first character. For full strings, we need:

✓ Correct version:

```
python
```

```
sorted(totalData.items(), key=lambda item: (-item[1][0], item[0]), reverse=False)
```

➔ This sorts by gold descending and name **ascending** — not what we want.

✅ Final solution (reverse only name):

We manually reverse the name using a trick — use `item[0]` negated by reversing the string:

python

```
sorted(totalData.items(), key=lambda item: (-item[1][0], item[0][::-1]))
```

✅ Explanation:

- `-item[1][0]` → Sort by gold (descending)
- `item[0][::-1]` → Reverse the country name to get reverse lexicographic behavior

Example:

python

Suppose we have:

```
totalData = {  
    'India': [2, 1, 0],  
    'Argentina': [2, 2, 1],  
    'Zimbabwe': [2, 0, 0],  
    'Brazil': [1, 1, 2]  
}
```

Sorting with:

```
sorted_data = dict(sorted(totalData.items(), key=lambda item: (-item[1][0], item[0][::-1])))
```

Output order:

- Zimbabwe (2 gold, z is last)
- India (2 gold, i)
- Argentina (2 gold, a)

- Brazil (1 gold)

Let me know if you'd like to also sort by silver and bronze in case of further ties!

↳ `item\[0]\[::-1]`

explain this

Sure! Let's break down what `item[0][::-1]` means in Python:

Step-by-step Explanation:

`item[0]`

This accesses the **country name**, because each item in `totalData.items()` is a (key, value) tuple, where:

- `key = item[0]` → the country name (e.g., "India")
- `value = item[1]` → a list of medal counts (e.g., [2, 1, 0])

`::-1]`

This is Python's **slicing syntax to reverse a string**.

- The full form of slicing is: `string[start:stop:step]`
- So `::-1]` means:
 - Start from the end of the string (`start` and `stop` are blank, so it defaults to the whole string)
 - Step backwards with `-1`

 Example:

python

```
"India"[::-1] # Output: "aidnI"
"Brazil"::-1] # Output: "lizarB"
```

 Why we use `::-1]` for reverse lexicographical order:

Python's `sorted()` sorts strings in **normal lexicographical order**, which is:

```
nginx
```

```
Argentina < Brazil < India < Zimbabwe
```

But for **reverse lexicographical**, we want:

```
nginx
```

```
Zimbabwe > India > Brazil > Argentina
```

So by reversing each string (`[::-1]`), Python sorts it based on the **backwards version**, which **flips** the lexicographic priority:

```
python
```

```
sorted(["India", "Argentina", "Zimbabwe", "Brazil"], key=lambda s: s[::-1])  
# → ["Zimbabwe", "India", "Brazil", "Argentina"]
```

And that's how `item[0][::-1]` helps us sort in **reverse dictionary order** ✓

Let me know if you want to visualize this with an example list!

it does the string reverse but, i want in descending lexicography.
according to your code 'abca' > 'zbba'
but what i want is z should come first

Ah! Got it now — thanks for pointing that out. You're **absolutely right**: using `item[0][::-1]` just reverses the string, it doesn't achieve **descending lexicographical order**. Let's fix this properly.

✓ Your goal:

Sort by:

1. Gold medals (descending)

2. If tied, sort by **country name in descending lexicographical order** (i.e., 'Zimbabwe' before 'India' before 'Argentina')

Correct approach:

Use the `sorted()` function and **invert the string sorting order** by using the `reverse` flag or **negate the string sort by sorting with `-gold`, then `country` in reverse**.

But we can't "negate" strings directly. So instead, we change the **order of sorting fields** and set `reverse=True` appropriately.

However, Python's `sorted()` does **not support per-key reverse**, so we do this manually:

Final working version:

python

```
sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], item[0]), # gold descending, country ascending
    reverse=False # So -gold stays descending
))
```

Now to get **country descending**, we reverse `item[0]` with a trick: negate it by **sorting by the string as-is but apply reverse order only to that part**.

Since Python doesn't let you reverse just the second key, we use a custom key where we **sort by `-gold`, then `country` reversed manually** using a negative comparison logic:

Hack with a tuple:

Reverse lexicographic order can be simulated by **sorting by the negative of the string**, which can be done by sorting it in reverse:

python


```
sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], -ord(item[0][0])) # ❌ This only checks the
    first letter
))
```

But that's not good. The proper fix is to just **invert the string by using a trick**:

✅ BEST SOLUTION:

Sort by gold descending, and **country descending** using the `item[0]` in descending order via `reverse=True`:

```
python

sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], item[0]),
    reverse=False # This keeps -gold descending, country ascending
))
```

But since we want **country descending**, we multiply the country name by `-1` equivalent.

🧠 Strings can't be negated, so we simulate this:

```
python

sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], item[0]),
    reverse=False
))
```

We flip the string sort direction by making a key that puts larger strings first:

```
python
```

```
sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], -ord(item[0][0])) # Not general
))
```

But this only works on first letters.

🔥 Final Working Code — Full Country Name Reverse:

The only solid way: sort by `-gold`, then **country descending**, so use:

```
python

sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], item[0]),
    reverse=False
))
```

Then **flip only the country name** manually by inverting string comparison:

✅ Final trick:

```
python

sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], ''.join(chr(255 - ord(c)) for c in item[0]))
))
```

💡 Why this works:

- `chr(255 - ord(c))` inverts the lexicographical order of each character
- So `"Zimbabwe"` becomes a string that sorts **after** `"India"`, etc.
- Effectively makes country sort in descending order

Example:

```
python

# Sample data
totalData = {
    'Argentina': [2, 2, 1],
    'India': [2, 1, 0],
    'Zimbabwe': [2, 0, 0],
    'Brazil': [1, 1, 2]
}

sorted_data = dict(sorted(
    totalData.items(),
    key=lambda item: (-item[1][0], ''.join(chr(255 - ord(c)) for c in item[0]))
))
```

✓ Output order:

1. Zimbabwe
2. India
3. Argentina
4. Brazil

Let me know if you want to sort by **gold**, then **silver**, then **bronze**, then **reverse country** too!

explain the argument parser thing in detail

Absolutely! The `argparse` module is Python's standard way to parse command-line arguments. It's super handy when you want to make your script configurable through terminal input — like passing file paths, numbers, flags, etc.

Let's break down your code and go deeper 🙌

🧩 Your code:

```
python
```

```
from argparse import ArgumentParser as ap

parser = ap()
parser.add_argument('--path', type=str, required=True)
args = parser.parse_args()
```

What this does:

1. `from argparse import ArgumentParser as ap`

- `argparse` is a built-in Python module to handle CLI arguments.
- You're importing `ArgumentParser` and giving it a short alias: `ap`.

So `parser = ap()` is the same as `parser = argparse.ArgumentParser()`.

2. `parser = ap()`

This creates a new `ArgumentParser` object:

- It will hold the description of the CLI arguments you want.
- You can give a description if you want:

```
python
```

```
parser = ap(description="Aggregate Olympic medals from files")
```

3. `parser.add_argument('--path', type=str, required=True)`

This line **defines a new command-line argument**.

Breakdown:

Argument	Meaning
<code>--path</code>	A named argument (optional in syntax, but you made it required). This is used like <code>--path ./medals</code> .
<code>type=str</code>	The argument must be a string.
<code>required=True</code>	The script will throw an error if <code>--path</code> isn't provided.

✓ So your script **must** be run like:

```
bash

python medals.py --path ./somefolder
```

If you forget `--path`, it will say:

```
lua

error: the following arguments are required: --path
```

4. `args = parser.parse_args()`

- This **parses the arguments** given when you run the script.
- It returns an object where each argument is accessible as a property.

So:

```
python

print(args.path)
```

Will give you whatever you typed with `--path`.

✓ **Full example:**

```
python
```

```
# medals.py
from argparse import ArgumentParser

parser = ArgumentParser(description="Medal counter")
parser.add_argument('--path', type=str, required=True, help='Path to the directory containing medal files')
args = parser.parse_args()

print("You provided:", args.path)
```

bash

```
$ python medals.py --path ./data
You provided: ./data
```

Bonus Tips:

Optional flags

You can add a flag like `--verbose`:

```
python

parser.add_argument('--verbose', action='store_true', help='Enable verbose output')
```

Then run with:

```
bash

python script.py --path ./data --verbose
```

And check it with:

```
python

if args.verbose:
    print("Verbose mode on!")
```

Positional arguments (no `--`)

If you want to allow:

```
bash
```

```
python script.py ./data
```

Use:

```
python
```

```
parser.add_argument('path', type=str)
```

This makes `path` **positional** instead of named.

Let me know if you want a visual demo of different argument types — like integers, choices, defaults, lists, etc.